Part 1 — K Nearest Neighbour (KNN)

Ek simple algorithm jo naye data point ke "sabse paas wale padosiyon" ko dekh kar prediction karta hai — classification aur regression dono ke liye.

1. Introduction — KNN Kya Hai?

K Nearest Neighbour (KNN) ek supervised machine learning algorithm hai jo **do kaam** kar sakta hai (teacher ne notes ke top par yahi do heading likhi hain):

- **Classification** — output ek category hota hai (jaise 0 ya 1, Yes/No).
- **Regression** — output ek continuous number hota hai (jaise house price).

Idea bahut simple hai: **"Tum apne dosto se pehchane jaate ho."** Naye data point ke aas-paas jo training points sabse nazdeek hain, unhi ke output ko dekh kar naye point ka output decide kar lete hain. "K" matlab **kitne padosi** dekhne hain.

> **Dhyaan dein:** KNN me koi khaas "training" nahi hoti — algorithm bas training data ko store kar leta hai. Asli kaam (distance nikalna, padosi dhoondhna) tab hota hai jab naya test data aata hai. Isliye isko **lazy learner** bhi kehte hain.

2. KNN Classification — Step by Step

Teacher ka setup: dataset me do features $\mathbf{f_1}$, $\mathbf{f_2}$ hain aur output **y** ek binary category hai (**0 ya 1**). Graph par 0 wale points white "x" aur 1 wale points red "x" ke roop me dikhaye gaye hain. Ab ek naya test point aata hai aur humein batana hai ki wo 0 category ka hai ya 1 ka.

Step 1 — K Ki Value Set Karo

Sabse pehle humein **K** decide karna hota hai, yaani kitne nearest neighbours dekhne hain. K koi bhi positive number ho sakta hai (K = 1, 2, 3, 4, 5, ...). K ek **hyperparameter** hai — matlab hum khud set karte hain, model apne aap nahi seekhta. Teacher ke example me **K = 5** hai.

Step 2 — Test Data Ke K Nearest Neighbours Dhoondho

Test point se **har training point ki distance** nikalo (distance kaise nikalte hain, wo section 3 me hai), aur un points ko chuno jinki distance sabse kam hai. K = 5 hai to **5 sabse paas wale** points.

Step 3 — Majority Vote Karo

Un K padosiyon me dekho ki kitne **0 category** ke hain aur kitne **1 category** ke. Jis category ke zyada hon, wahi test point ki prediction.

K	0 category wale padosi	1 category wale padosi	Prediction
5	2	3	**1** (kyunki 3 > 2)

> **Yaad rakhein:** Notes ke example me K = 5 neighbours me 2 zeros aur 3 ones aaye, isliye test point ko **1** label mila. Classification me K aam taur par **odd number** rakhte hain (jaise 3, 5, 7), taaki do category me tie na ho.

3. Distance Kaise Nikalte Hain? — Euclidean aur Manhattan

"Sabse paas" ka matlab hai sabse kam **distance**. Notes me do popular tareeke diye gaye hain.

3.1 Euclidean Distance

Do points ke beech ki **seedhi (straight line) doori** — jaise kauwa udta hua jaata hai. Points (x_1, y_1) aur (x_2, y_2) ke liye:

$$\textbf{Distance} = \sqrt{[\ (x_2 - x_1)^2 + (y_2 - y_1)^2\ ]}$$

3.2 Manhattan Distance

Isme seedhi line nahi, balki **grid par chalna** hota hai — pehle ek direction me, phir doosri me, jaise city ke blocks me chalte hain (teacher ne notes me gali/blocks ka drawing banaya hai). Formula me **absolute differences ka sum** hota hai:

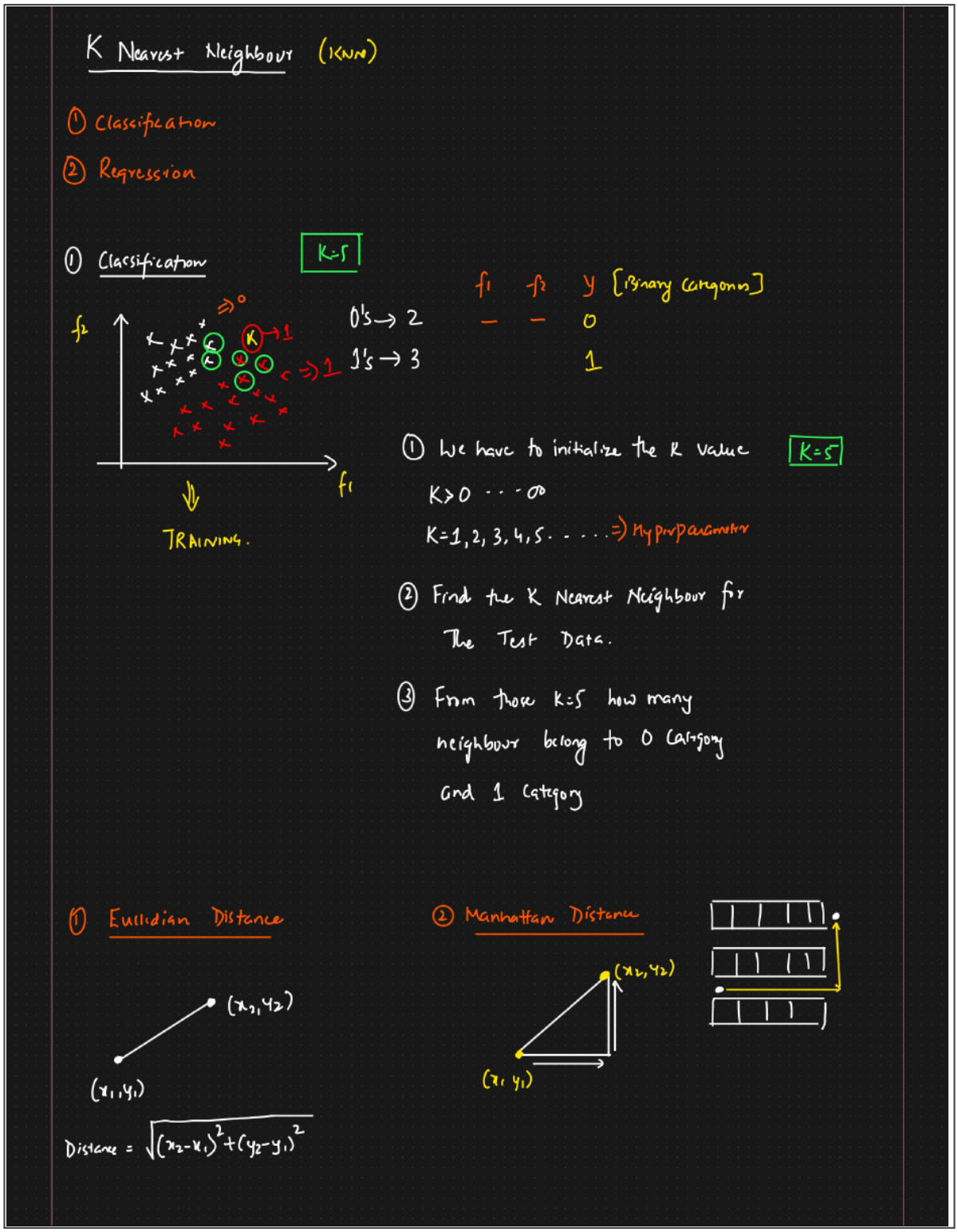
$$\textbf{Distance} = |\ x_2 - x_1\ | + |\ y_2 - y_1\ |$$

Chhota example (samajhne ke liye): points (1, 2) aur (4, 6) lo.

Distance type	Calculation	Answer
Euclidean	$\sqrt{[\ (4-1)^2 + (6-2)^2\ ]} = \sqrt{(9+16)} = \sqrt{25}$	**5**
Manhattan	$\|\ 4-1\ \| + \|\ 6-2\ \| = 3 + 4$	**7**

> **Dhyaan dein:** Manhattan distance hamesha Euclidean se bada ya barabar hota hai, kyunki seedhi line sabse chhota raasta hai. (Extra: in dono ko jodne wala general formula **Minkowski distance** kehlata hai; p = 2 par Euclidean aur p = 1 par Manhattan.)

Original Handwritten Notes — KNN, Page 1



Isme dikhaya gaya hai: KNN ke do use (Classification, Regression), classification ka K = 5 wala example (2 zeros, 3 ones → class 1), KNN ke 3 steps, aur Euclidean tatha Manhattan distance ke formulas.

4. KNN Regression

Jab output ek **continuous number** ho, tab KNN regression use hota hai. Teacher ka example: **House Price vs Size**. Graph me x-axis par size aur y-axis par price hai, aur har cross ek ghar hai.

Process classification jaisa hi hai, bas last step alag hai:

- **Step 1:** K choose karo.
- **Step 2:** Naye ghar (test point) ke K nearest neighbours dhoondho (blue circle ke andar wale points).
- **Step 3:** Majority vote ki jagah in K padosiyon ke prices ka **average (mean)** nikalo — wahi predicted price hai.

Notes me teacher ne "Median" bhi likha hai, yaani average ki jagah **median** bhi liya ja sakta hai. Agar data me kuch bahut bade ya bahut chhote (outlier) prices hon, to median zyada safe rehta hai kyunki wo extreme values se kam hilta hai.

Extra example: maan lo K = 3 aur teen nearest gharon ke price 50, 60 aur 70 lakh hain. To mean = (50 + 60 + 70) / 3 = **60 lakh** (median bhi 60 lakh).

Point	Classification	Regression
Output	Category (0 / 1)	Number (price)
K padosiyon se kya karte hain	Majority vote	Mean (ya median)

5. Time Complexity Ki Problem

KNN me jab bhi koi naya test point aata hai, humein usse **training ke har point** tak ki distance nikalni padti hai. Agar training me **n** points hain to har prediction ke liye **O(n)** kaam lagta hai (features bhi hon to O(n × d)). Teacher ne likha hai: **O(n)** **Million of data points** — yaani agar data me lakhon-crore points hain, to har ek prediction bahut slow ho jaayegi.

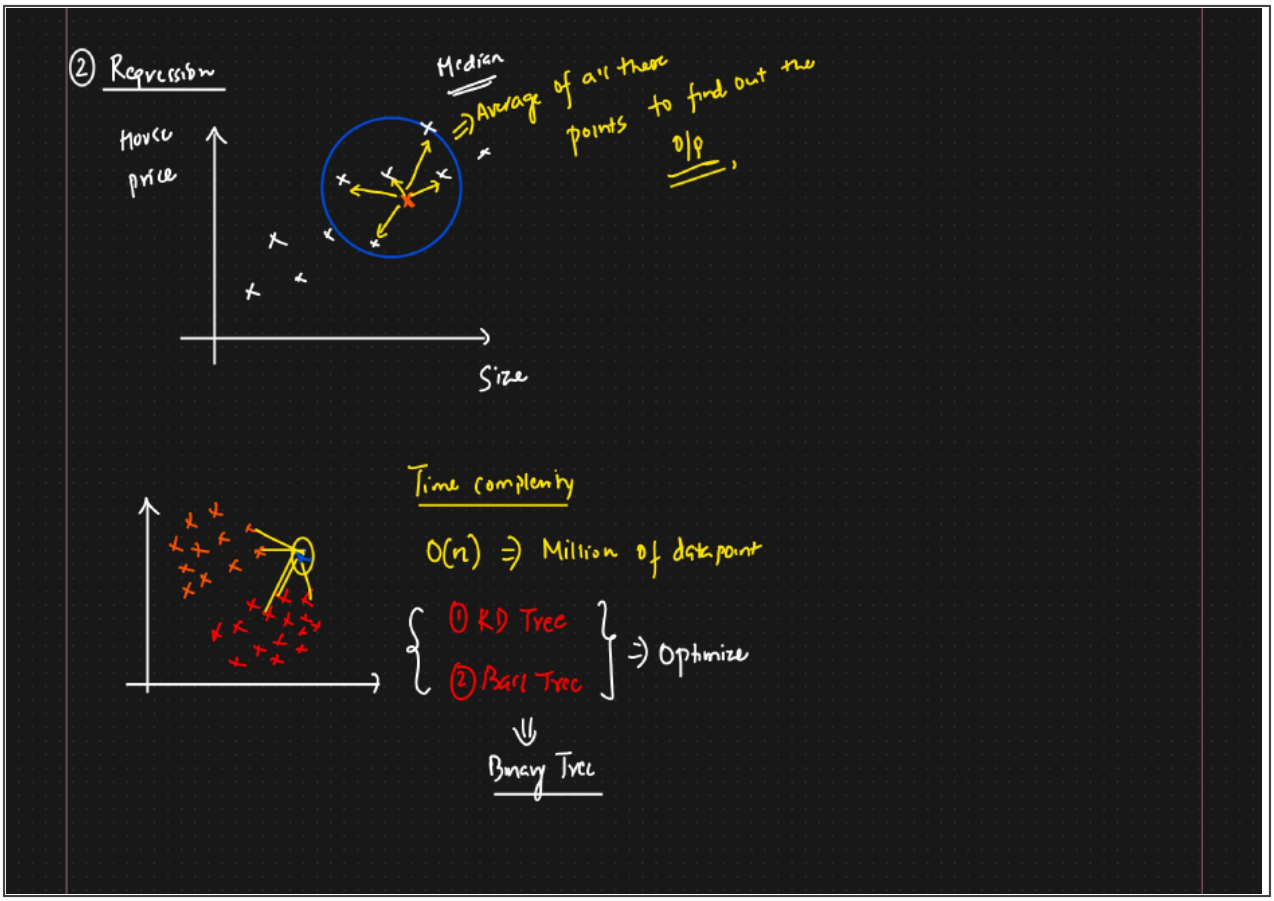
Isko tez karne (optimize) ke liye do variants use hote hain, jo dono **Binary Tree** ka idea use karte hain:

- **KD Tree**
- **Ball Tree**

Binary tree me data ko aise tod diya jaata hai ki naye point ke liye humein **poora data scan nahi karna padta**, sirf kuch hi hisse dekhne padte hain. Time complexity **O(n) se ghat kar roughly O(log n)** tak aa sakti hai. Ye dono Part 2 me detail me hain.

> **Yaad rakhein:** KNN ki sabse badi kamzori hai **slow prediction** bade dataset par. KD Tree aur Ball Tree isi problem ka solution hain.

Original Handwritten Notes — KNN, Page 2



Isme dikhaya gaya hai: KNN Regression (House Price vs Size, K padosiyon ka average/median), aur Time Complexity O(n) ki problem jisko KD Tree aur Ball Tree (Binary Tree) se optimize kiya jaata hai.

Part 2 — KNN Variants: KD Tree aur Ball Tree

Bade dataset par nearest neighbours jaldi dhoondhne ke liye data ko tree structure me organize karne ke do tareeke.

6. Idea — Tree Kyun Chahiye?

Brute-force KNN me test point ko har ek training point se compare karna padta hai. Tree-based variants me pehle **training data ko ek binary tree me arrange** kar lete hain. Phir test point aane par hum tree me **neeche ki taraf sahi raaste** se chalte hain aur jo hisse bahut door hain, unko skip kar dete hain. Isse time complexity bahut kam ho jaati hai (notes me "Time Complexity ↓↓" likha hai).

7. KD Tree

KD Tree (k-dimensional tree) ek binary tree hai jisme har level par data ko **kisi ek feature ki median value** par do hisso me baant dete hain. Har level par feature badalta rehta hai (pehle f_1, phir f_2, phir wapas f_1 ...).

7.1 Dataset

Teacher ke example me 6 points hain, jinke features (f_1, f_2) ye hain:

f1	f2
7	2
5	4
9	6
2	3
4	7
8	1

7.2 Tree Banane Ke Steps

Level	Kis feature par split	Median kaise nikala	Node ban gaya
0 (root)	f_1	f_1 sorted: 2, 4, **5, 7**, 8, 9. Beech ke do (5 aur 7) ka average = 12/2 = 6.5. Isliye f_1 = 7 wala point root liya.	**(7, 2)**. Left: $f_1 < 7$ → (5,4), (2,3), (4,7). Right: $f_1 > 7$ → (9,6), (8,1)
1 (left)	f_2	Left group ke f_2: 3, **4**, 7. Median = 4.	**(5, 4)**. Iske neeche (2,3) ($f_2 < 4$) aur (4,7) ($f_2 > 4$)
1 (right)	f_2	Right group ke f_2: 1, 6. Dono me se (9,6) ko node liya.	**(9, 6)**. Iske neeche (8,1) ($f_2 < 6$)

Dhyaan dein: Notes me f_2 ka median poore data ka bhi likha hai (1, 2, **3, 4**, 6, 7 → 3.5). Lekin tree me doosre level par har **subset** (left group / right group) ka apna median liya jaata hai, jaisa upar table me hai. Jab number even ho (jaise 6 points), to beech ke do values me se ek point ko node bana lete hain.

Final tree ka structure:

Root	Left child	Right child	Leaf nodes
(7, 2)	(5, 4)	(9, 6)	(2, 3), (4, 7) under (5,4); (8, 1) under (9,6)

7.3 Nearest Neighbour Search Aur Backtracking

Ab test point aata hai. Notes ke graph me test point approx **(5, 7)** par dikhaya gaya hai. Hum is par 1 nearest neighbour (K = 1) nikalte hain:

- **Neeche jao:** root (7,2) par f_1 compare karo: $5 < 7$ → left. Node (5,4) par f_2 compare karo: $7 > 4$ → right → leaf **(4, 7)**.
- Ye pehla **candidate** hai. Distance = $\sqrt{[(5-4)^2 + (7-7)^2]} = \mathbf{1}$. Best distance abhi 1 hai.
- **Backtracking (wapas upar):** ab tree me wapas upar chadhte hain aur check karte hain ki doosri taraf koi aur paas ka point to nahi chhoot gaya.
- Node (5,4): iski splitting line $f_2 = 4$ hai. Test point se line ki doori $|7 - 4| = 3$ hai, jo best distance 1 se zyada hai → doosri branch **skip**.
- Root (7,2): splitting line $f_1 = 7$. Doori $|5 - 7| = 2 > 1$ → poora right subtree (9,6), (8,1) bhi **skip**.
- Final answer: nearest neighbour = **(4, 7)**, aur humein sirf kuch hi points dekhne pade, saare 6 nahi.

Yaad rakhein: Backtracking rule: doosri branch tabhi check karo jab test point se splitting line ki doori, ab tak ki best distance se **kam** ho. Warna us poori branch ko skip kar sakte ho — yahi tree ki speed ka raaz hai.

8. Ball Tree

Ball Tree bhi ek binary tree hai, lekin ye data ko lines se nahi balki **gol "balls" (hyperspheres)** me group karta hai. Har ball ka ek **center** aur ek **radius** hota hai, jisme uske saare points aa jaate hain.

8.1 Tree Kaise Banta Hai (Notes Ka Example)

Teacher ke diagram me 1 se 9 tak numbered points hain, jinhe pehle chhote groups me baanta gaya hai, phir groups ko milakar bade groups banaye gaye hain:

Group	Isme kya hai	Level
G1	points 1, 2	Sabse chhota ball
G2	points 3, 4	Sabse chhota ball
G3	points 5, 6, 7	Sabse chhota ball
G4	points 8, 9	Sabse chhota ball
G5	G1 + G2	Bada ball
G6	G3 + G4	Bada ball
G7	G5 + G6 (poora data)	Root

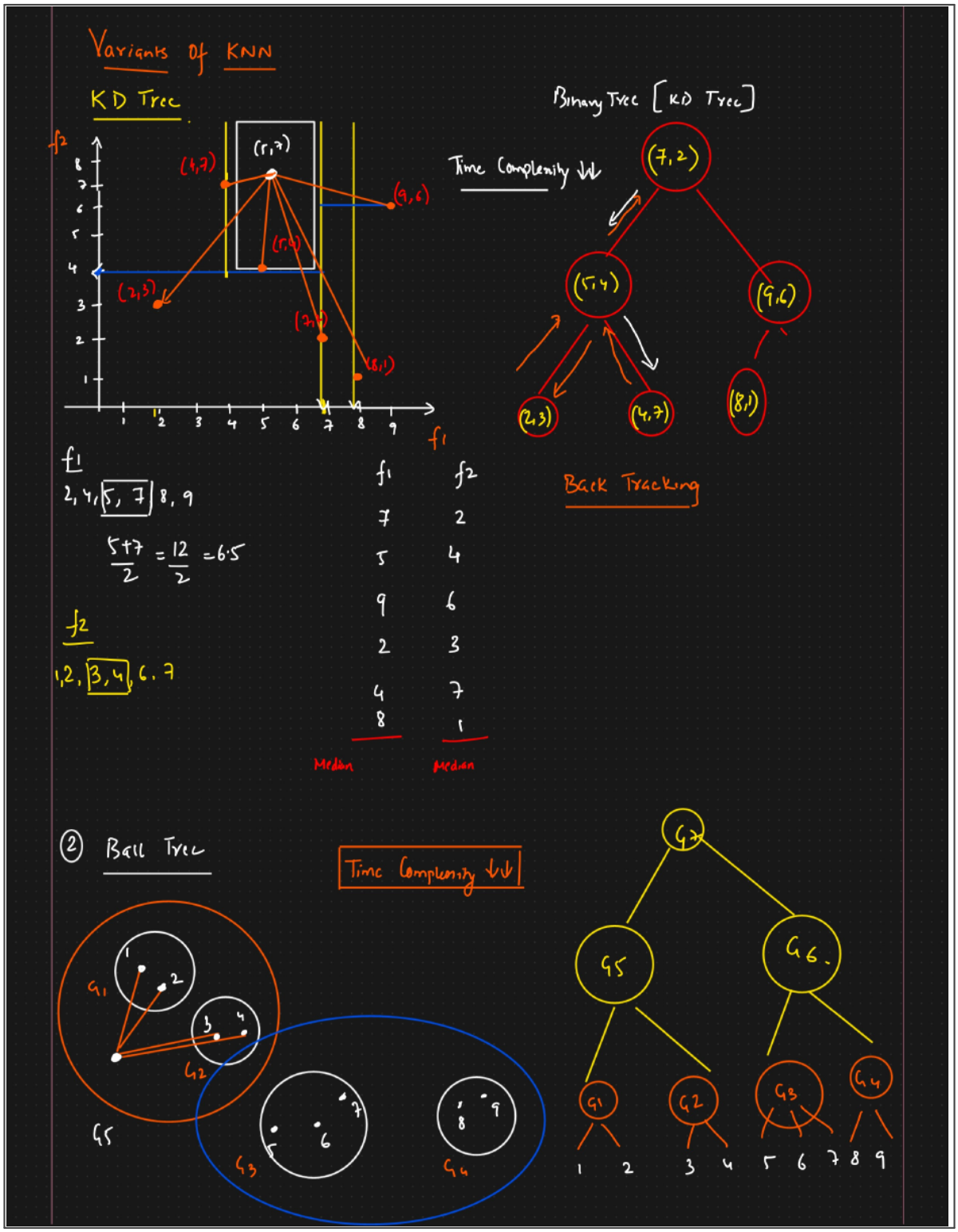
Yaani tree **neeche se upar** samajh sakte ho: leaf par points hain, unke upar chhote balls (G1–G4), unke upar bade balls (G5, G6), aur sabse upar root ball G7.

8.2 Search Kaise Hota Hai?

- Test point se **ball ke center tak ki distance** nikalo. Us ball ke andar kisi bhi point ki minimum possible distance = **(center tak distance − radius)** hogi.
- Pehle usi ball me neeche jao jo test point ke sabse paas ho (notes me G5 → G1/G2) aur wahan se ek best candidate nikalo.
- Ab doosre balls ke liye check karo: agar kisi ball ki minimum possible distance current best se bhi zyada hai, to us **poore ball ko skip** kar do.
- Notes ke example me G3 aur G4 (blue circle, G6) test point se door hain, isliye unke saare points ko individually check karne ki zaroorat nahi padti.

> **Dhyaan dein:** KD Tree me skip karne ka faisla **splitting line** ki doori se hota hai, jabki Ball Tree me **ball ke center aur radius** se. Isliye Ball Tree **high-dimensional data** me aam taur par KD Tree se behtar chalta hai.

Original Handwritten Notes — KD Tree & Ball Tree



Isme dikhaya gaya hai: KD Tree ka dataset, f1 aur f2 ke median, binary tree (7,2) → (5,4), (9,6) → leaves, Backtracking ka arrow, aur Ball Tree me G1–G4 balls se G5, G6 aur root G7 ka tree. Dono ke saath "Time Complexity ↓↓" likha hai.

9. Brute Force vs KD Tree vs Ball Tree

Point	Brute Force	KD Tree	Ball Tree
Structure	Koi nahi, har point check	Binary tree, median par split	Binary tree, balls (center + radius)
Search time (aam taur par)	O(n)	Roughly O(log n)	Roughly O(log n)
Kab achha	Chhota dataset	Kam features (low dimension)	Zyada features (high dimension)
Skip kaise karta hai	—	Splitting line ki doori se	Ball ki center aur radius se

10. Bonus — Practical Tips

(Ye section teacher ke notes me nahi hai, lekin practical aur exam dono me kaam aata hai.)

- **Feature scaling zaroori hai:** KNN distance par chalta hai, isliye agar ek feature ki range bahut badi ho (jaise salary) aur doosre ki chhoti (jaise age), to bada feature distance par haavi ho jaayega. StandardScaler ya MinMaxScaler lagao.
- **K ka asar:** bahut chhota K (jaise 1) noise par overfit kar sakta hai; bahut bada K underfit kar deta hai. Best K cross-validation se chuno.
- **Scikit-learn me:** KNeighborsClassifier / KNeighborsRegressor ka **algorithm** parameter "auto", "kd_tree", "ball_tree" ya "brute" ho sakta hai; **n_neighbors** hi K hai.

11. Poore Notes Ka Quick Revision

Topic	Yaad rakhne wali baat
KNN kya hai	Naye point ke K sabse paas wale training points dekh kar prediction
K	Hyperparameter (K = 1, 2, 3, ...); notes me K = 5
Classification	K padosiyon me majority vote (2 zeros vs 3 ones → 1)
Regression	K padosiyon ka mean (ya median)
Euclidean	$\sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$
Manhattan	$\lvert x_2 - x_1 \rvert + \lvert y_2 - y_1 \rvert$
Problem	Brute force me time complexity O(n) — million data points par slow
KD Tree	Median par split karne wala binary tree; search me backtracking
Ball Tree	Points ko balls (center + radius) me group karne wala binary tree

> **Ek line me yaad rakhna:** KNN = **"K sabse paas wale padosiyon ki raay"** (classification me vote, regression me average). Data bada ho to KD Tree ya Ball Tree se search tez karo.